Part 1: Problem Framing and Dataset Analysis

The data set includes three datasets: yellow_tripdata (a raw trip dataset of 7,667,792 trips for January 2019); taxi_zone_lookup (a 265 zones dataset); and taxi_zones spatial metadata (a shapefile dataset converted to GeoJSON). We had to deal with three main issues: negative fares, trips less than 30 seconds, or more than 24 hours, out of range distances, and a tip_percentage DECIMAL overflow due to the fact that we have near zero base fares.

The most surprising was the pickup timestamps that were set for 2088 — they were wrong both as a pickup timestamp and as a dropoff timestamp, which is why they didn't get caught by the duration checks. It was added with this specific purpose given by a dedicated year-range filter (2009-2024).

A total of 58,724 rows (0.8%) were rejected, each exclusion being written to a log file in excluded_record_log with a reason. Three features were engineered during ingestion: trip_duration_min (dropoff - pickup duration in minutes), avg_speed_mph (dropoff - pickup distance in miles / minutes), and tip_percentage (tip / fare / 100), in addition to an is_airport_trip flag that was added when a trip's pickup or dropoff address contained an airport.

2. System Architecture and Design Decisions

The system has three layers: a plain HTML/CSS/JS front end, a Node.js/Express REST API, and a MySQL database. Both cleaning and loading are done via a one-off ingestion pipeline.

Layer	Technology	Responsibility
Frontend	HTML / CSS / JS + Chart.js	Dashboard, filters, charts, paginated trips table
Backend	Node.js + Express	REST API, query building, algorithm execution
Database	MySQL 8	Star schema, indexing, efficient querying
Ingestion	Node.js scripts	Cleaning, feature engineering, batch inserts

The schema is based on the triangle table, which has two foreign keys into the zone dimension table, (zone_pickup and zone_dropoff) zone_boundary stores the polygons in GeoJSON format separately so that there is no unnecessary pull of spatial data used in normal queries. The monetary columns don't involve floating point, so DECIMAL is used. The indexes are pickup_datetime, both location IDs, total_amount and trip_distance_mi. The rate codes and payment types are stored on the trip table as integers, instead of in lookup tables, as this is considered a trade-off (less joins) for strict 3NF at this scale.

3. Introduction to Algorithmic Logic and Data Structures.

Custom HashMap and quicksort were also created in src/algorithms/zoneAggregation.js without using any built-in functions to group or sort (no Map, Array.sort, lodash, etc.).

Custom Hash Map

In a single O(n) pass, groups all trips by their pickup_location_id and calculates the number of trips per zone, total revenue, total distance, and tip percentage per zone. Implements Knuth's multiplicative hash function: key × 2654435761 mod 2^{32} and then mod bucketCount for collision resolution using chaining. Automatically resizes if load factor is greater than 0.75. Time: O(n) average. Space: O(m) where m = distinct zones (~265).

Custom Quicksort (Lomuto Partition)

Ranks the ~265 zone aggregates by a chosen metric (trip_count, total_revenue, avg_distance). Lomuto scheme: last element used as pivot, single left-to-right scan sorts elements into those smaller than pivot, and those larger than pivot. Supports a compareFn to sort for ascending or descending. Time: O(m log m) on average and O(m^2) worst case — acceptable as m is not too big ($m \leq 265$). Time Complexity: O(log m) call stack.

Pseudocode: quicksort(arr, lo, hi) is called when lo<hi: then pivot = partition(arr, lo, hi); quicksort(arr, lo, pivot−1); quicksort(arr, pivot+1, hi).

4. Insights and Interpretation

Insight 1: Demand builds up during commuting hours and midnight.Insight 1: There is a surge in demand at the commuting hours and during the midnight.

This is calculated by calling /api/insights/hourly-demand (SQL GROUP BY HOUR). There are two prominent peaks at 8-9 AM and 6-8 PM, corresponding to commuting to work; a less distinct bump later on during the night around midnight, which is possibly due to Manhattan nightlife. The slowest time is between 3 am and

5 am. These three windows should be treated as non-uniform demand in these cases and hence dynamic pricing should aim at these windows instead of assuming a uniform demand.

Insight 2: Manhattan sees the highest number of pickup cars.

Produced using the custom hash map + quicksort (top-zones endpoint) and SQL borough aggregation. The top 10 pickup zones are Manhattan — Midtown Center, Upper East Side, Times Square. The lack of outer-borough pickups is explained by the restrictions imposed by TLC on yellow cab street hails to just Manhattan and airports.

Insight 3: Cash tip data is not reliable.

Calculated using tip_percentage data after filtering by payment_type. The average tips for credit card travel ranged from 18% to 22% by borough. Cash trips appear as 0% — this is not because the passengers don't tip, but because the TLC meter system is unable to track the cash tips. Tip reporting should only be analyzed in the context of transactions that are made with credit cards. Aggregated tip statistics across all payments are not reliable.

5. Suggest ideas for further research.

- The ingest pipeline had to be rewritten during build from parquetjs-lite to csv-parser, due to the trip file being delivered with a .parquet extension. This was a contained change because ingest.js was broken up into modules.

- Initial ingestion failed because of memory exhaustion, CSV stream was sending events faster than MySQL could send them out. Stream.pause / stream.resume - this is the way to handle it so that memory doesn't become too big.

- In top-zones, all the rows are loaded into application memory to execute the custom algorithm – a deliberate choice for DSA demo, not suitable for production. Future work: Ensure that a pre-aggregated rollup table is kept up-to-date on ingestion.

- A known limitation mentioned in the codebase: only client-side filtered for 200 rows in a sample of 7.6M rows, rather than server-side. Future work: Use Borough as a server-side SQL filter parameter.

- Future enhancements: Leaflet.js choropleth map using GeoJSON stored boundaries; multi-month dataset for seasonality trend analysis; authentication and rate limiting prior to any public deployment.